

Exercise 8 — Performance Optimization Challenge

Exercise Description

Select a performance scenario, analyse it with the "Identifying Performance Bottlenecks" prompt, implement the optimisation, **measure before and after**, and document the process, results, and learnings.

Scenario chosen: Slow Code Analysis (Python) — `find_product_combinations` in `inventory_analysis.py`, which finds product pairs whose combined price is within `target ± margin`.

Files: - `inventory_analysis_original.py` — the starter code (unchanged). - `inventory_analysis_optimized.py` — the optimised implementation. - `benchmark.py` — correctness + timing harness.

1. Why the original is slow

```
for i in range(len(products)):
    for j in range(len(products)):
        if i != j:
            ...
            if in_range(combined):
                if not any(r['product1']['id']==product2['id'] and
                           r['product2']['id']==product1['id'] for r in results):
                    results.append(...)
```

Two problems: 1. **$O(n^2)$ double loop** over all *ordered* pairs (it checks both (a,b) and (b,a)). 2. **$O(R)$ duplicate scan inside the inner loop** — for every matching pair it linearly scans the entire `results` list so far to reject the mirror pair. When many pairs match (R large), this makes the real cost roughly **$O(n^2 + R^2)$** . With `target=500`, `margin=50` and prices 5-500, a large fraction of pairs match, so R is huge and this term dominates.

2. The optimisation

Sort by price once, then for each product binary-search only the partners whose price falls in the valid window, pairing only `j > i` so each unordered pair is produced exactly once (no dedup scan needed):

```
sorted_products = sorted(products, key=lambda p: p['price'])
prices = [p['price'] for p in sorted_products]
for i in range(n):
```

```

lo = (target - margin) - prices[i]
hi = (target + margin) - prices[i]
start = bisect_left(prices, lo, i + 1)
end = bisect_right(prices, hi, i + 1)
for j in range(start, end):
    results.append({...})

```

Complexity: $O(n \log n)$ sort + $O(n \log n + R)$ search/collection. The expensive inner dedup scan is eliminated entirely.

3. Measured results (from `benchmark.py`)

n	Original	Optimized	Speedup
250	2.389 s	0.0038 s	~630×
400	15.902 s	0.0115 s	~1385×
5000	did not finish (>40 s)	10.3 s (2.42M pairs)	—

- **Correctness:** verified identical unordered-pair sets between the two implementations (e.g. 8,806 pairs at $n=300$). See the `[correctness ...]` line printed by `benchmark.py`.
- The speedup **grows with n** ($630\times \rightarrow 1385\times$), confirming the original's super-linear-in-results blow-up.
- At $n=5000$ the optimised version spends its ~ 10 s almost entirely **constructing 2.42M result dicts**, not on the algorithm. If callers only need the closest matches, returning a generator or top-N would cut this dramatically.

4. Reflection Questions

1. **How did optimisation change your understanding?** The headline " $O(n^2)$ " wasn't the real killer — the **inner linear dedup scan** was. Removing duplicate work by construction (pair only $j > i$) mattered more than the loop order.
2. **Were the improvements significant enough to justify the change?** Overwhelmingly — 100s-to-1000s× with identical output and simpler code (no dedup logic).
3. **What did you learn about bottlenecks you didn't know?** Cost can be dominated by the *number of results* (R), not just input size (n); always consider R when a function emits many items.
4. **How would you approach similar issues in future?** Measure first, identify whether cost scales with n or R , remove repeated work by construction, and use sorting + binary search to replace nested scans.
5. **Tools to find such issues proactively?** `cProfile`/`timeit` for hotspots, `line_profiler` for per-line cost, and reasoning about output cardinality (R) up front.

Submission for Exercise 8 — Performance Optimization Challenge

1. Scenario selected: Slow Python code — `find_product_combinations`.

2. Analysis of the issue: $O(n^2)$ ordered-pair loop **plus** an $O(R)$ duplicate scan inside the inner loop → roughly $O(n^2 + R^2)$ when many pairs match.

3. Optimisation implemented: sort by price + `bisect` to select valid partners, pairing only `j>i` to avoid duplicates — $O(n \log n + R)$. See [inventory_analysis_optimized.py](#).

4. Before/after measurements: $\sim 630\times$ at $n=250$ and $\sim 1385\times$ at $n=400$ (gap widens with n); original doesn't finish at $n=5000$ while the optimised version does in ~ 10 s (dominated by building 2.42M result objects). Output verified identical. See [benchmark.py](#).

5. Process, results & key learnings: measure → find the true hotspot (the dedup scan, not just the nested loop) → eliminate duplicate work by construction → replace linear partner search with binary search; the biggest lesson is that result cardinality (R) can dominate runtime, and reflection answers are given above.